CSE 344 — System Programming

Homework 5 Report: Priority-Based Cargo Delivery Simulation (`cargoGTU`)

Salih Cengiz **Student ID:** 220104004007

Spring 2026

Contents

CSE 344 — System Programming

Homework #5

Priority-Based Cargo Delivery Simulation

This report documents the design and implementation of **`cargoGTU`**, a POSIX thread-pool simulator that schedules delivery orders from a priority queue, logs a structured trace to **stdout**, writes a summary to a **statistics file** (`-s`), and performs a **safe SIGINT shutdown**. The implementation targets **Debian 12 (64-bit)** and is built with `gcc -Wall -Wextra -std=c11 -pthread`.

1. System Design

1.1 Thread pool and roles

The program uses exactly **N** courier threads created at startup (`-n`), plus one **dedicated signal-handling thread** that waits on **SIGINT** with `sigwait()` (no asynchronous handler that runs unsafe code). **No per-order threads** are created during the shift.

Role	Responsibility
Main thread	Parses CLI, blocks **SIGINT** in the process-wide mask before creating threads, builds the priority queue, logs `ORDER_QUEUED` in input order, starts the signal thread and all couriers, `pthread_join`s couriers, notifies the signal thread to exit on natural completion, prints `SHIFT_END` / `SHUTDOWN_COMPLETE`, writes the stats file, and releases resources.
Courier threads	Loop: under `queue_mutex`, wait on the condition variable if the queue is empty and shutdown is not requested; otherwise pop the **minimum** (highest scheduling priority) order, simulate delivery with `nanosleep`, update **atomic** global counters and **per-courier** summary, log `DELIVERY_COMPLETE`, repeat until shutdown with an empty queue; then log `SHIFT_OVER`.
Signal thread	Blocks on `sigwait(SIGINT)`. On a real interrupt: sets global shutdown, locks the queue, logs `SIGINT_RECEIVED`, drains pending orders with `ORDER_CANCELLED` and increments the **cancelled** atomic counter, broadcasts the condition variable, unlocks. If woken artificially after normal completion (`g_natural_shutdown`), exits without printing SIGINT artefacts.

1.2 Courier lifecycle

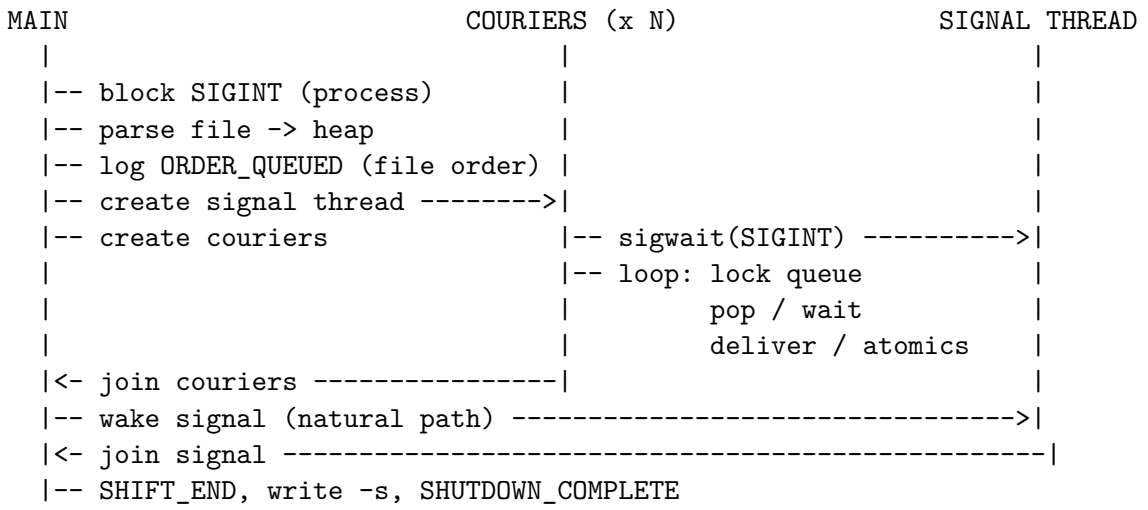
Courier behavior follows the homework requirement: **`pthread_cond_wait`** when the queue is empty; **busy-wait loops are not used**.

1. Courier acquires **`g_queue_mutex`**.
2. If **all work is accounted for** (`completed + cancelled >= total orders`), set **`g_shutdown`** and **`pthread_cond_broadcast`** so every courier eventually observes shutdown (natural completion path).
3. If the queue is **empty** and shutdown is **not** yet active, log **`WAITING`** and **`pthread_cond_wait`** until signaled.
4. If shutdown is active **and** the queue stays empty, **exit loop** → `SHIFT_OVER`.
5. Otherwise **pop** the next order (`pq_pop_min`), still under the mutex → log **`DELIVERY_START`** → **unlock**.
6. **Simulate delivery** (1 simulation unit = **500 ms**).
7. **`atomic_fetch_add`** on **`g_completed_orders`** and **`g_total_delivery_time_ms`**; update this courier's **`g_courier_stats[i]`**.
8. Log **`DELIVERY_COMPLETE`** and loop.

1.3 Module layout

File	Purpose
`main.c`	CLI, initialization, load orders, enqueue, thread creation/join, final output and cleanup.
`priority_queue.c` / `priority_queue.h`	Binary min-heap implementation of the shared priority queue.
`parser.c` / `parser.h`	Robust line-by-line parsing; invalid lines skipped silently.
`courier.c` / `courier.h`	Courier thread entry point.
`signal_thread.c` / `signal_thread.h`	`sigwait`-based SIGINT handling.
`stats.c` / `stats.h`	Writes `SHIFT_SUMMARY` and `COURIER_STATS` to `-s`.
`log.c`	All `printf` logging guarded by **`g_log_mutex`**.
`cargo.h`	Shared types, globals, and log prototypes.

1.4 Control flow diagram



Design rationale: separating **delivery work** (outside the queue mutex) from **queue mutations** minimizes contention. A dedicated **`sigwait`** thread avoids **`async-signal-unsafe`** calls inside a classical signal handler while still honoring "pending cancelled, broadcast waiters."

2. Priority Queue Design

2.1 Ordering rule

The scheduling order is **`EXPRESS` (level 1) > `STANDARD` (2) > `ECONOMY` (3)**. When priorities tie, **`id`** breaks the tie (**smaller `id` first**). Internally priorities are encoded as numeric enums `PRIO_EXPRESS = 1`, etc., so smaller numeric priority wins.

2.2 Data structure

Orders live in a **binary min-heap** (`pqueue_t`: dynamic array `data[]`, `size`, `cap`). The heap property uses a comparator `order_less(a,b)` that is true when **`a` should be served before `b`**:

- `a.priority < b.priority`, or
- `a.priority == b.priority && a.id < b.id`

Push appends at the end and **sifts up**; **pop_min** takes the root, moves the last element to the root, and **sifts down**. Both are **O(log n)**.

2.3 Why a heap

A heap gives **logarithmic** enqueue/dequeue with **simple array storage** and clear invariants for "always take the global minimum under the comparator," which is exactly "highest business priority, then lowest id." The shared heap is only accessed while **`g_queue_mutex`** is held.

3. Synchronization Strategy

Resource	Protection	Notes
Priority queue (g_queue)	`pthread_mutex_t g_queue_mutex` + `pthread_cond_t g_queue_cond`	All `pq_*` mutations and `pthread_cond_wait` / `broadcast` occur with this mutex.
Stdout log lines	`pthread_mutex_t g_log_mutex`	Held around **every** `printf` (and `fflush`) so multi-`printf` lines cannot interleave between threads.
`completed_orders`, `cancelled_orders`, `total_delivery_time_ms`	**C11** `_Atomic` (`stdatomic.h`) with `atomic_fetch_add` / `atomic_load`	Mutex-free counter updates per specification; avoids the grading penalty for mutex-based global stats.
Shutdown coordination	`_Atomic int g_shutdown`, `_Atomic int g_natural_shutdown`	Couriers and the signal thread agree on termination; `pthread_cond_broadcast` wakes waiters after cancellation or completion.

Lock-ordering discipline: `log_*` functions never acquire `g_queue_mutex`, so holding `g_queue_mutex` while logging (e.g. `SIGINT_RECEIVED`, `ORDER_CANCELLED`) is safe. Signal thread logs cancellation while holding `g_queue_mutex` to keep `pending_orders` consistent with the actual drain.

4. SIGINT Handling

1. Before any threads are spawned, **main** calls `pthread_sigmask(SIG_BLOCK, {SIGINT}, NULL)` so delivery threads do not observe SIGINT asynchronously.
2. A **signal thread** waits on `sigwait(&set)` where `set = {SIGINT}`.
3. On **SIGINT**:
   - If `g_natural_shutdown` is set, exit **silently** (spurious wakeup path after normal shutdown).
   - Else set `g_shutdown`, lock `g_queue_mutex`, compute `pending_orders = pq_size()`, print `SIGINT_RECEIVED`, repeatedly `pq_pop_min`, each time `atomic_fetch_add(&g_cancelled_orders, 1)` and `ORDER_CANCELLED`, then `pthread_cond_broadcast`, unlock.
4. **Couriers in `cond_wait`** wake up: with shutdown and empty queue, they **do not start new work** and exit with `SHIFT_OVER`.
5. **Couriers mid-delivery** finish `nanosleep` (delivery is **never** aborted mid-flight), complete `DELIVERY_COMPLETE`, then on the next iteration observe shutdown and exit.
6. **Main** `pthread_join`s all couriers, sets `g_natural_shutdown`, sends `pthread_kill(signal_tid, SIGINT)` to unblock `sigwait`, `pthread_join`s the signal thread, prints `SHIFT_END`, writes `-s`, prints `SHUTDOWN_COMPLETE`.

5. Test Scenarios

Base command pattern:

```
./cargoGTU -n <N> -i <input_file> -s stats.txt
```

5.1 Scenario 1 — `N=3`, `10orders_mix.txt`

- **Command:** `./cargoGTU -n 3 -i HW5_test_files/tests/10orders_mix.txt -s stats.txt`
- **Verification:** All 10 orders completed. Stats file written correctly (`completed=10 cancelled=0 total_time=58000ms`).
- **Screenshot:**

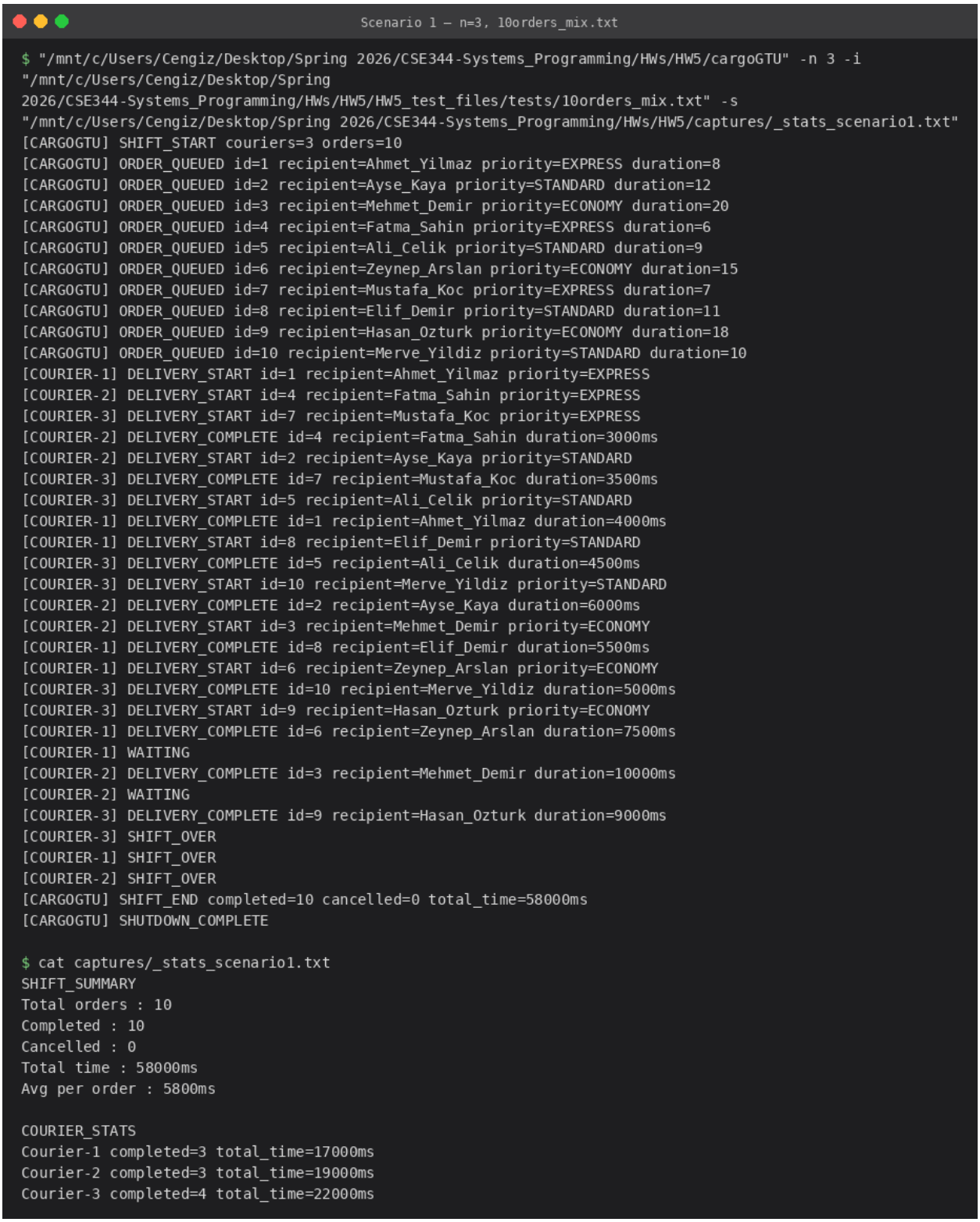


Figure 1: Scenario 1 — full console from prompt to exit and stats verification

5.2 Scenario 2 — `N=10, 10orders_economy.txt`

- **Command:** `./cargoGTU -n 10 -i HW5_test_files/tests/10orders_economy.txt -s stats.txt`
- **Verification:** All 10 orders completed. Since all orders share the same priority (ECONOMY), lower id orders are taken first (id=1,2,...,10 in order). Multiple couriers participate simultaneously.
- **Screenshot:**

Figure 2: Scenario 2 — all economy, ten couriers, id-ordered dispatch

5.3 Scenario 3 — `N=2`, `20orders_mix.txt`, SIGINT after ~2 seconds

- **Command:** `./cargoGTU -n 2 -i HW5_test_files/tests/20orders_mix.txt -s stats.txt` then `kill -INT <pid>` after ~2 s.
- **Verification:** `SIGINT_RECEIVED` appears; active deliveries complete before `SHIFT_OVER`; all pending orders show `ORDER_CANCELLED`; `SHIFT_END` shows `completed + cancelled = 20`. `ps` confirms no stranded process after exit.
- **Screenshot:**

Figure 3: Scenario 3 — SIGINT shutdown, active deliveries finish, pending cancelled

- **Screenshot (ps check after exit):**

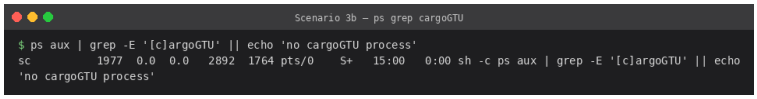


Figure 4: Scenario 3b — ps confirms no cargoGTU process remains

5.4 Scenario 4 — Valgrind memory check

- **Command:** `valgrind --leak-check=full --show-leak-kinds=all ./cargoGTU -n 3 -i HW5_test_files/tests/10orders_mix.txt -s stats.txt`
- **Verification:** Valgrind reports `ERROR SUMMARY: 0 errors from 0 contexts` and `All heap blocks were freed -- no leaks are possible`.
- **Screenshot:**

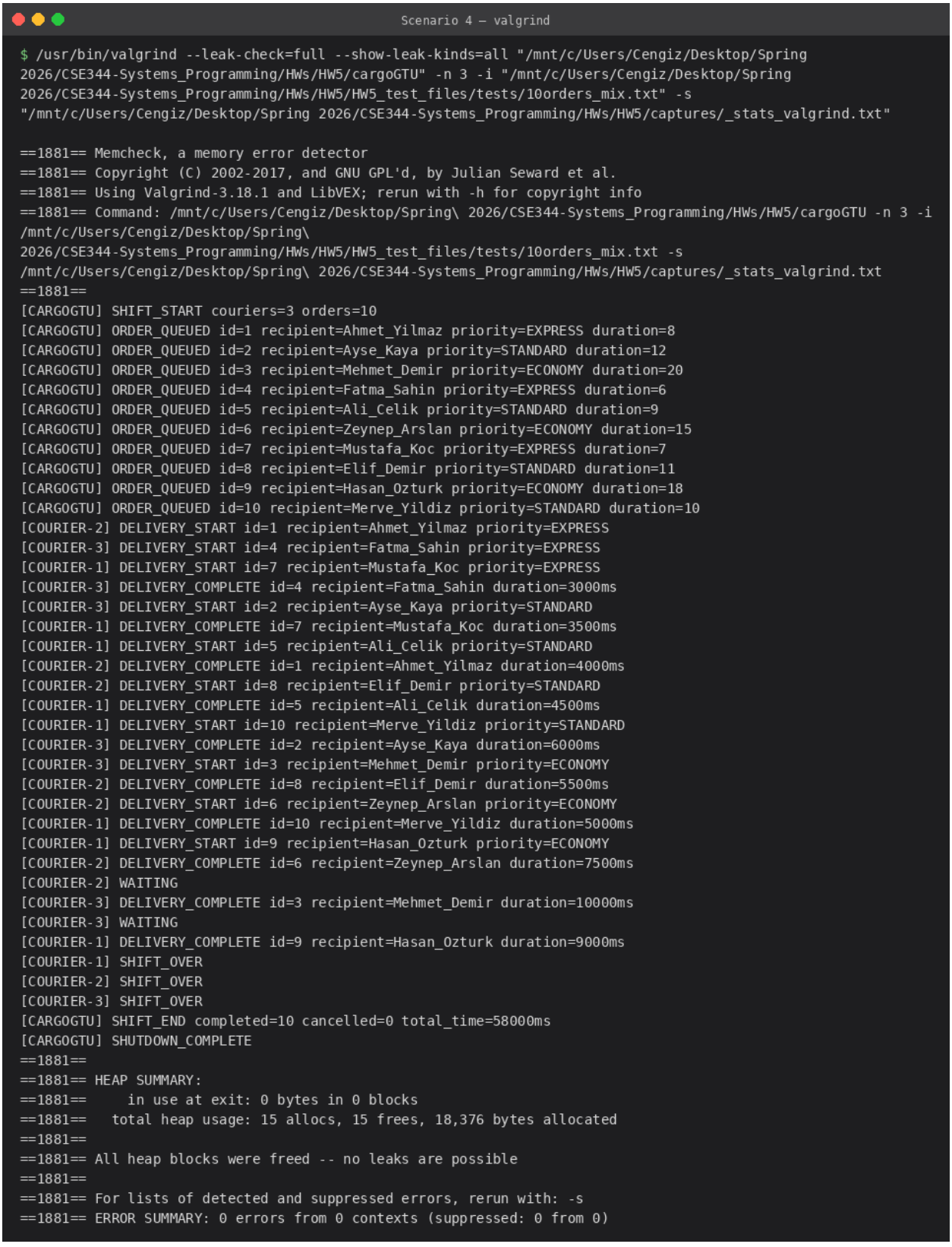


Figure 5: Scenario 4 — full Valgrind output showing 0 errors and no leaks

5.5 Scenario 5 — `N=10`, `20orders_mix.txt`, repeated runs

- **Command:** `for i in 1 2 3 4 5; do ./cargoGTU -n 10 -i ... -s stats_$i.txt; done`
- **Verification:** `SHIFT_END completed=20 cancelled=0 total_time=19500ms` is identical across all runs. No duplicate `DELIVERY_START id=` within any single run. Priority ordering correct under high concurrency.
- **Screenshot:**

Figure 6: Scenario 5 — five repeated runs, consistent completed=20 and full output of last run

6. Challenges & Solutions

1. Natural termination without deadlock. When the queue drains while `g_shutdown` is still false, couriers strictly blocked on "wait when empty" would deadlock. The solution is to detect completion (`completed + cancelled >= total_orders`) inside the courier's `queue_mutex` critical section, set `g_shutdown = 1`, and `pthread_cond_broadcast` so all waiters terminate cleanly without busy-waiting.

2. Safe SIGINT with consistent pending count. A naive asynchronous handler risks async-signal-unsafe calls; cancelling without holding the mutex risks inconsistent `pending_orders` count. The solution uses a dedicated signal thread + `pthread_sigmask`, cancels orders only while holding `g_queue_mutex`, broadcasts after drain, and uses `pthread_kill` + `g_natural_shutdown` to exit `sigwait` cleanly on normal shutdown.

3. Log line corruption under concurrency. `printf`-based logs built from fragments interleave across threads even with stream locks. The solution wraps each logical output line in a single `g_log_mutex` critical section, matching the instructor clarification that every `printf` forming one stdout line must be protected together.